Lecture 6: Spatial Heterogeneity and Calibration

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 18, 2026

University of Toronto

A Spatial Aiyagari

Setting: three locations

- Three locations `loc1`, `loc2`, `loc3`.
- Each produces **the same single good**: a costlessly-tradable, perfectly-substitutable numeraire.
- Each has its own **productivity** A_j: $A_1 > A_2 > A_3$.
- Local production: $Y_j = A_j K_j^{\alpha} L_j^{1-\alpha}$.

Households

- Each household has one unit of inelastic labor.
- **State** this period: (b, j) — wealth + current location.
- **No** idiosyncratic income shocks. Within j, every household earns w_j.
- All heterogeneity is over (b, j).

One bond market

- **One asset**, traded freely across the three locations and with the rest of the world.
- Capital flows freely $\Rightarrow$ **MPK equalized**: $r + \delta = \alpha A_j (L_j/K_j)^{1-\alpha}$.
- Wages adjust: $w_j = (1-\alpha) A_j \left(\frac{\alpha A_j}{r+\delta}\right)^{\alpha/(1-\alpha)}$.

Where does r come from?

- **Small open economy.** The country is *small* in the world bond market $\Rightarrow$ r is taken as given (exogenous, $r = 0.03$).
- **Why exogenous today?** Without idiosyncratic income shocks, household saving is weak; endogenous r would sit at the saving boundary $1/\beta - 1$. Exogenous r lets us focus on **migration + calibration** without bond-market fragility.
- **Trade-off:** the asset market is degenerate; everything else gets a clean treatment.

Within-period chain

1. **Migration.** Draw a Gumbel taste shock over destinations. Pick j', pay cost $C[j, j']$, get amenity bonus $\alpha_{j'}$.
2. **Wealth update.** At destination j': $b' \leftarrow (1 + r)b + w_{j'}$.
3. **Consumption / savings.** Choose next-period wealth.

The chain (compact form):

$$\text{Migr.} \circ_s \text{WealthCh.} \circ_s \text{ConsSav.}$$

What we want to know

Steady-state objects:

- Population shares s_j across locations.
- Wealth distribution $\Lambda(b, j)$ (joint).
- Aggregate $K_{\text{total}} = \sum_b b\,\Lambda(b, \cdot)$.
- Value function $V(b, j)$.

Prices (r, w) — where $w = (w_1, w_2, w_3)$ — are pinned down before the household solve.

MigrationStage

The Gumbel choice problem

At origin i, the per-destination payoff is

$$u_{ij} = -C[i,j] + \alpha_j + V_{\text{end}}(j,b),$$

- plus an idiosyncratic **Gumbel taste shock** $\varepsilon_j \sim$ EV1 (i.i.d. over j).
- The household picks $j^\star = \arg\max_j \left(u_{ij} + \eta \cdot \varepsilon_j\right)$, where η is the shock **scale**.

The log-sum-exp formula

- With $\varepsilon_j \sim$ EV1, two classical facts:
- **(a) Choice probabilities are logit:**

$$\Pr(j \mid i, b) = \frac{\exp(u_{ij}/\eta)}{\sum_k \exp(u_{ik}/\eta)}.$$

(b) Expected value is the inclusive value:

$$\mathbb{E}\Big[\max_j (u_{ij} + \eta \cdot \varepsilon_j)\Big] = \eta \log \sum_k \exp\big(u_{ik}/\eta\big).$$

Why log-sum-exp matters

- $\max$ is non-smooth, non-differentiable at ties.
- $\eta \log \sum \exp(\cdot/\eta)$ is **smooth** in its arguments.
- As $\eta \to 0$: recovers $\max$ (the argmax wins).
- As $\eta \to \infty$: all choices equally likely (random allocation).
- η is a **smoothing** / preference-noise parameter.

The `MigrationStage` kernel

Backward pass (computes V_{pre}):

$$V_{\text{pre}}(b, i) = \eta \log \sum_j \exp \frac{-C[i, j] + \alpha_j + V_{\text{end}}(b, j)}{\eta}.$$

Forward pass (pushes Λ):

$$\Lambda_{\text{post}}(b, j) = \sum_i \Pr(j \mid i, b)\, \Lambda_{\text{pre}}(b, i).$$

Same kernel as the static logit, threaded through the chain by V/Λ duality.

The API
API

```
migration = MigrationStage(layout;
    migration_cost = C,                          # (n_loc, n_loc) origin → dest
    amenity        = (dest; env) -> envα.[loc_idx(dest)],ε
                   = pη._logit,                  # Gumbel scale η
)
```

- Cost is a static matrix; the amenity closure reads α off `env` on every backward pass.
- The Greek-kwarg `ε` is the stage's name for the Gumbel scale; we pass `p.η_logit` into it.
- Stage allocates the $(\text{cells}, n_{\text{loc}})$ probability tensor itself.

Preference shifters α_j

A per-destination utility add-on α_j enters the migration payoff additively:

$$u_{ij} = -C[i,j] + \alpha_j + V_{\text{end}}(j,b).$$

- $\alpha = (\alpha_1, \alpha_2, \alpha_3)$ lives on env (not the Spec), so we vary it just by rebuilding env. Normalization $\alpha_1 \equiv 0$ (only differences are identified).
- The calibration loop in §5 will iterate on α to match observed shares.

The household block in code

```
layout = StateLayout(
    StateAxis(:wealth,   continuous_grid(0, 25; length=250, spacing=:log)),
    StateAxis(:location, categorical(collect(LOC_NAMES))))

amenity = (dest; env) -> envα.[loc_idx(dest)]
migration = MigrationStage(layout; migration_cost=p.C_base, amenity, ε=pη._logit)
income = WealthChangeStage(layout; wealth_post=function(cell; env)
    return (1 + env.r) * cell.wealth + env.w[loc_idx(cell.location)]
end)
savings = ConsumptionSavingsStage(layout; β=pβ.,
    utility=(cell, c; env) -> u_crra(c, Val(pσ.)))

hh = migration ∘ income ∘ savings
define_moments!(hh;
    K_total = at_end(integrand=(cell; env) -> cell.wealth, reduce=sum),
    L       = at_end(integrand=(cell; env) -> Float64[cell.location == j for j in LOC_NAMES],
         reduce=sum))
```

Steady State

A new kind of outer loop

- L4: outer loop on $\bar{K}$ (Aiyagari market clearing).
- L5: outer loop on the path $\{K_t\}$ (transitions).
- **L6: outer loop on α (calibration).** r is exogenous; $w_j(r, A_j)$ closed-form. The outer iteration matches population shares.
- Each outer step is one `solve_steady_state_given_env!` call.

Inner solve

```
function solve_household(hh, p, α; V_init=nothing, Λ_init=nothing)
    env = make_env(hh; r=p.r, w=spatial_wage.(p.r, p.A, p), α)
    (;V, Λ, moments, history) = solve_steady_state_given_env!(hh, env; V_init, Λ_init)
    shares = moments.L ./ sum(moments.L)
    return (; V, Λ, env, moments, shares, history)
end
```

V backward to fixed point at env, then Λ forward to stationarity. Then read moments.

Demo run at the initial guess $\alpha = 0$

Before the calibration loop starts. With $r = 0.03$, $A = (1.20, 1.00, 0.85)$, $\alpha = (0, 0, 0)$:

- Wages: $w = (1.66, 1.25, 0.97)$.
- Shares: $(0.39, 0.37, 0.25)$.
- $K_{\text{total}} = 0.75$.

At $\alpha = 0$, `loc1`'s high productivity makes it the most attractive destination; `loc3`'s low wage drives population out. The calibration will fix that.

Reading V across locations

At fixed wealth b:

$$V(b, j) \gtrless V(b, j')$$

- purely from wage differences and the cost structure $C[i, j]$.
- **Migration choice** comes from comparing $V_{\text{end}}(j, b')$ at the wealth b' implied by moving to j.
- (Implementation: the `MigrationStage` kernel populates `choice_prob`, a $(N_w, 3, 3)$ tensor.)

Calibration

Calibration: model meets data

- **Familiar example (L4):** pick β so that the model's K/Y matches the data K/Y. One parameter, one moment.
- **Today:** pick **preference shifters** $\alpha = (\alpha_1, \alpha_2, \alpha_3)$ so that the model's **population shares** match the data shares.
- Three parameters, three moments. (Normalization $\alpha_1 = 0$ leaves two free; shares sum to 1 leaves two binding.)

The data we want to match

Synthetic target:

$$s^{\text{data}} = (0.30,\ 0.30,\ 0.40).$$

- The data say `loc3` houses *40%* of households — even though its productivity (and wage) is *lowest*.
- **Something keeps people in `loc3`** that the $\alpha = 0$ model does not see: an "amenity," a non-pecuniary preference for `loc3`. The shifter α_3 will quantify it.

The static intuition

In a **static** multinomial logit with deterministic-part utilities δ_j:

$$s_j = \frac{\exp(\delta_j/\eta)}{\sum_k \exp(\delta_k/\eta)}.$$

Take log differences:

$$\log s_j - \log s_{j'} = \frac{\delta_j - \delta_{j'}}{\eta}.$$

If δ_j contains α_j additively, we recover $\alpha_j - \alpha_{j'}$ directly from observed log-share differences.

A damped log-share update

The static formula motivates a damped update on α:

$$\alpha_j^{(k+1)} = \alpha_j^{(k)} + s \cdot \eta \cdot \left(\log s_j^{\text{data}} - \log s_j^{\text{model},(k)}\right),$$

with update speed $s \in (0, 1]$. In our *dynamic* model, V also depends on α, so this is not an exact inversion — but it remains an empirically conservative fixed-point iteration with geometric convergence.

Why this is "indirect"

- **Direct inference:** read off the parameter from the data. We can't — we don't observe α_j in any data set.
- **Indirect inference (broad sense):** use the *model* as a translator: observe shares (which we have) and back out α (which we don't) by matching.
- **Note.** Strict Gourieroux–Monfort–Renault (1993) uses an auxiliary model. Macro talk uses the term more loosely; we follow the loose usage.

The calibration loop

1. Initialize $\alpha^{(0)} = 0$.
2. **Solve the household** at $\alpha^{(k)}$. Get $s^{\text{model},(k)}$.
3. Compute gap $g_j = \log s_j^{\text{data}} - \log s_j^{\text{model},(k)}$.
4. If $\|g\|_\infty < \text{tol}$, **stop**.
5. Update $\alpha^{(k+1)} = \alpha^{(k)} + s \cdot \eta \cdot g$, normalize $\alpha_1 = 0$.
6. Repeat.

In code

```
function calibrate_shifters!(hh, p, s_data; verbosity=1)
    update_speed = 0.1;  tol = 5e-3;  maxiter = 60α
     = Float64[0.0, pα._init[1], pα._init[2]];  V, Λ = nothing, nothing
    for k in 1:maxiter
        out = solve_household(hh, p, α; V_init=V, Λ_initΛ=);  (;V, Λ) = out
        gap = log.(s_data) .- log.(max.(out.shares, 1e-8))
        maximum(abs.(gap)) < tol && return (; α, k, out...)α
         .+= update_speed * pη._logit .* gap; α   .-= α[1]   # update + normalize
    end
end
```

Running it

Iteration log (selected, $s = 0.1$):

- Iter 1: shares (0.387, 0.365, 0.248); $\alpha = 0$.
- Iter 8: shares (0.327, 0.329, 0.343); $\alpha = (0, 0.024, 0.504)$.
- Iter 16: shares (0.306, 0.311, 0.383); $\alpha = (0, 0.010, 0.678)$.
- Iter 32: shares (0.300, 0.301, 0.398); $\alpha = (0, -0.011, 0.737)$. **Done.**

Converges in 32 iterations. Geometric convergence in the log-share gap.

Reading the calibrated α

- $\alpha_1 = 0$ (normalized).
- $\alpha_2 \approx -0.011$: `loc2`'s 30% share is consistent with the wage-only model.
- $\alpha_3 \approx 0.737$: `loc3` has ≈ 0.74 utility per period of unobserved amenity.

$\alpha_3 \approx 0.74$ is the kind of inference indirect identification gives us: a value for a quantity we cannot directly see.

What if data shares were different?

Comparative static of identification.

- If s_3^{data} rose from 0.40 to 0.45, α_3 would rise to keep the model honest.
- If s_3^{data} fell to 0.20 (less than the wage-only prediction), α_3 would go *negative* — a *dis*amenity that the wage model fails to capture.

The inferred α **tracks the data**, not the model's preferences. That's what makes it a useful empirical object.

Extensions

What our model assumed

We assumed:

1. **One good**, perfectly substitutable across locations.
2. **Costless trade** in that good.
3. **Free capital** flow (one r).
4. **Free labor** flow (subject to migration cost).

These are strong. Spatial economics tracks what changes when each is relaxed.

No code today — just the headline.

Variation 1: differentiated goods (Armington)

Each location's good is a *distinct variety*; consumers love variety (CES aggregator).

- Trade flows from variety demand.
- Local prices p_j differ; real wages depend on the local price index.
- Migration responds to *real* wages.

This is the **Armington (1969)** structure underlying modern trade / spatial economics (Eaton–Kortum, Allen–Arkolakis).

Variation 2: trade costs

Goods don't cross borders for free: **iceberg cost** $\tau_{ij} \geq 1$ (you must ship τ_{ij} units for 1 to arrive).

- Local prices depend on distance.
- Some goods become non-tradable ($\tau \to \infty$): services, housing.
- Households consume a mix of tradables and non-tradables.
- Migration costs and trade costs interact (closer regions tend to share both).

Variation 3: restricted factor mobility

- **Capital is local** (e.g. housing). Local capital scarcity raises local rental rates and house prices.
- **Migration cost is sunk and large.** Workers don't equalize wages quickly; spatial wage differentials persist.
- **Skill heterogeneity.** High- and low-skill workers face different migration choices — as in modern urban literature (Diamond 2016, Moretti 2011).

Putting the pieces back

Each variation slots into the chain as a *stage* or a *block-level extension*:

- Armington / non-tradables $\Rightarrow$ *change* the env (add p_j, real wages).
- Local housing $\Rightarrow$ *add an axis* (housing) and a stage.
- Skill heterogeneity $\Rightarrow$ *add an axis* (skill) and stage closures keyed on it.